

Student Details

Group 103 - Conversational AI (AIMLCZG521)

Student name	Student ID
Kodhandan S	2024ac05203
Gowrishankar S	2024ac05046
Arunkumar K A	2024ac05045
Naveen R	2024ac05219

Sequence-to-Sequence Transformer for Abstractive Text Summarization

Assignment Overview

Develop a complete Sequence-to-Sequence Transformer model from scratch for abstractive summarization using the CNN/DailyMail dataset.

Task Breakdown

- Task 1: Content Truncation & Cleaning (1.5 marks)
- Task 2: Subword Tokenization & Dynamic Padding (1.5 marks)
- Task 3: Encoder-Decoder & Cross-Attention Block Construction (2 marks)
- Task 4: Causal & Sequence Masking Implementation (1.5 marks)
- Task 5: Pre-training Loop with Causal Cross-Entropy (2 marks)
- Task 6: Inference Decoding & ROUGE Benchmarking (1.5 marks)

Dataset

CNN/DailyMail Dataset from HuggingFace: https://huggingface.co/datasets/cnn_dailymail

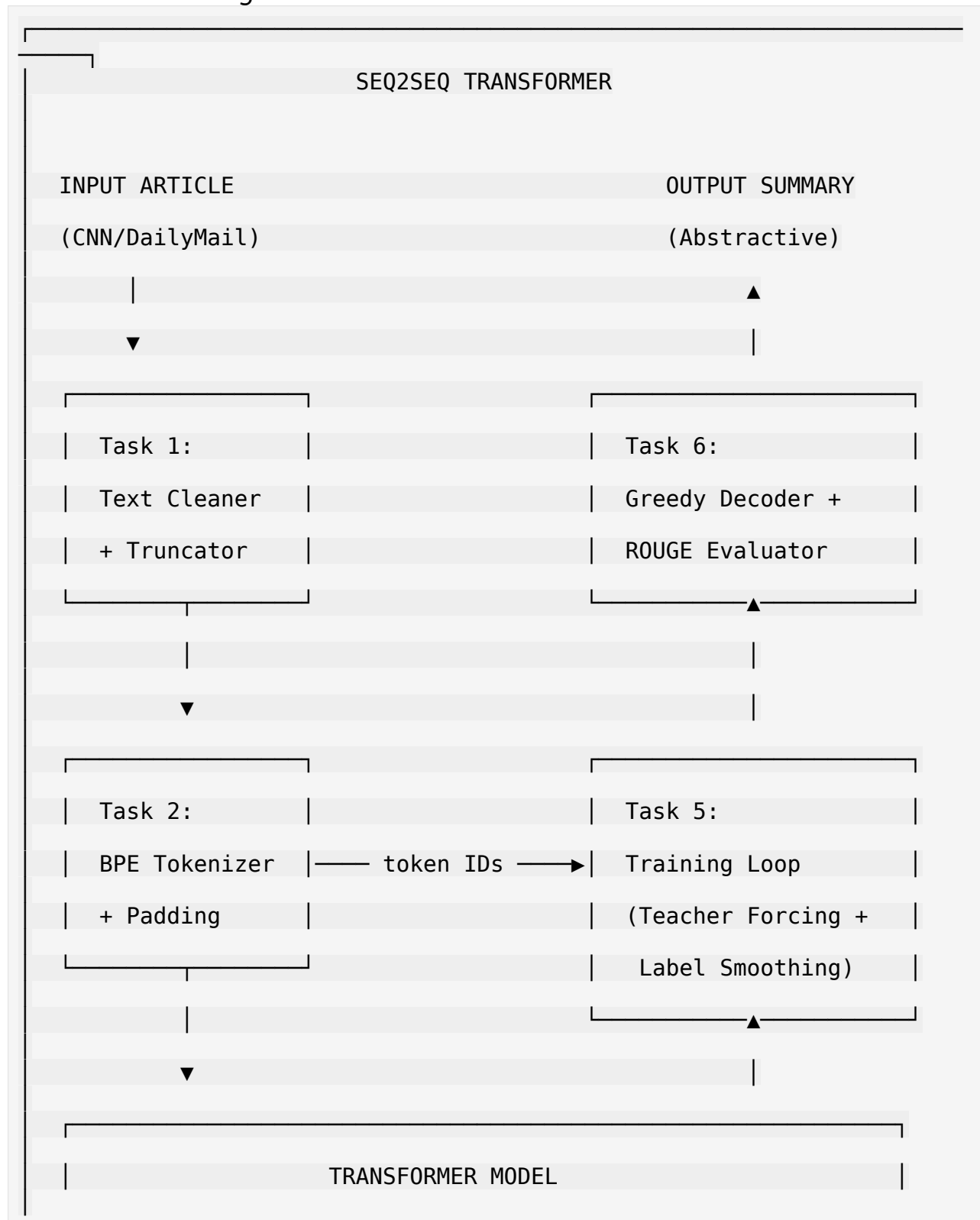
- Long-form news articles paired with human-written summaries
- Used for abstractive summarization research
- ~287K training examples

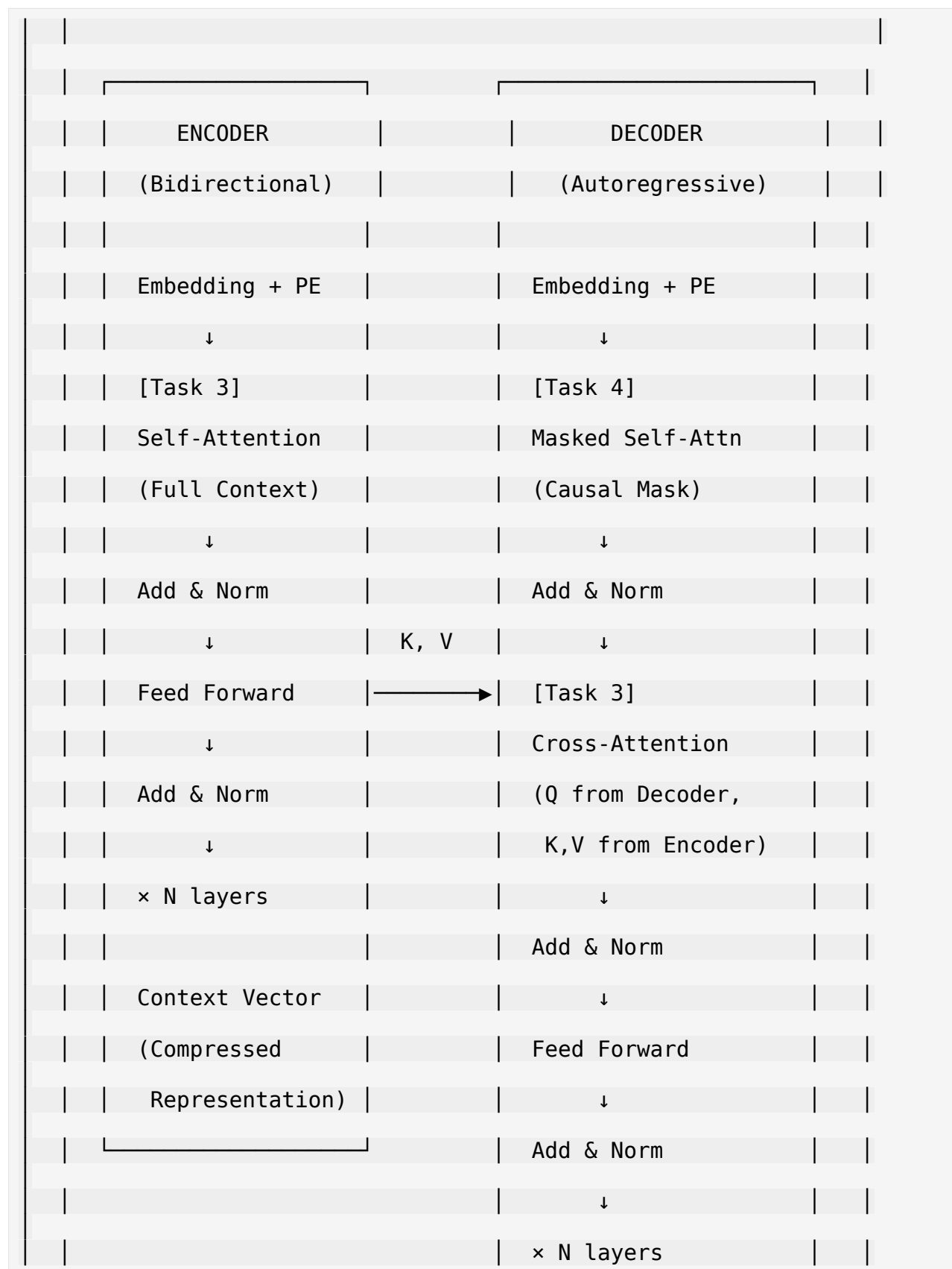
System Design

Overview

This assignment implements a **Sequence-to-Sequence (Seq2Seq) Transformer** for abstractive text summarization, trained end-to-end on the CNN/DailyMail dataset. The design follows the original "Attention Is All You Need" (Vaswani et al., 2017) encoder-decoder architecture, adapted to the news summarization domain.

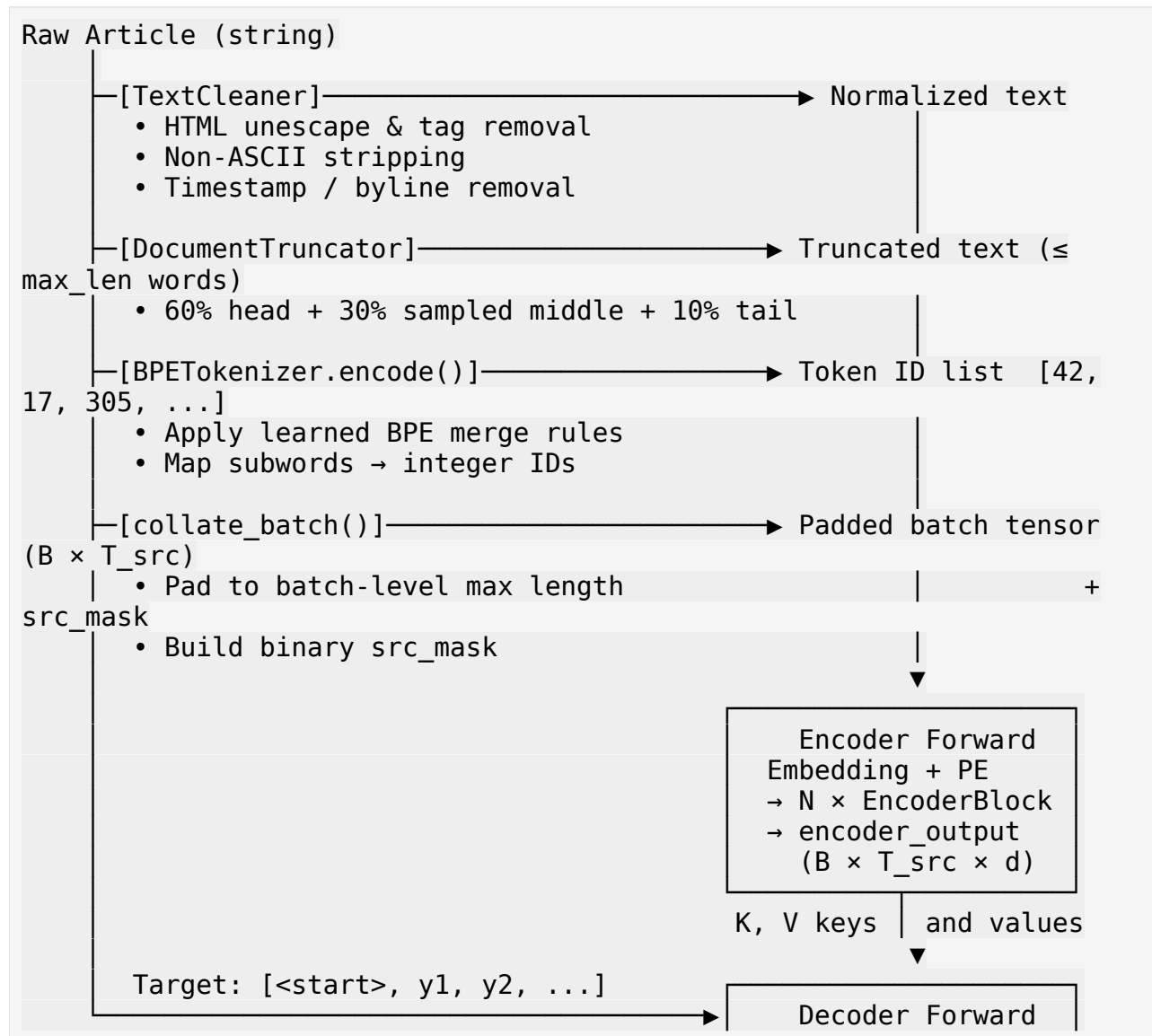
Architecture Diagram

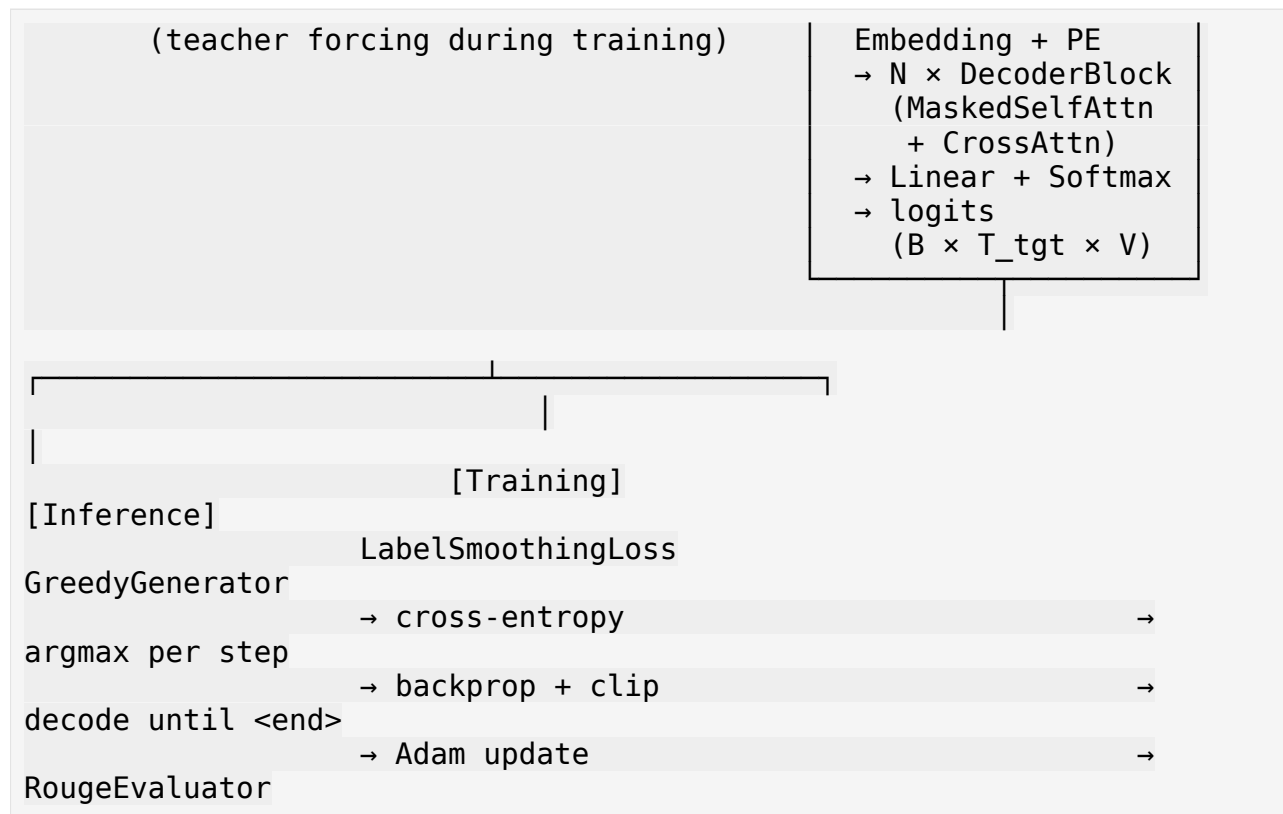




Component	Class / Function	Design Responsibility
Trainer	<code>Trainer</code>	Teacher forcing loop with gradient clipping
Generator	<code>GreedyGenerator</code>	Autoregressive argmax decoding at inference time
Evaluator	<code>RougeEvaluator</code>	ROUGE-1, ROUGE-2, ROUGE-L (F1) computed from scratch
Error Analyzer	<code>ErrorAnalyzer</code>	Detects repetition loops, hallucination risk, length pathology

Data Flow





Key Design Decisions

1. Encoder-Decoder over Decoder-Only

The task involves a **highly asymmetric length ratio**: source articles (~800 tokens) → target summaries (~50 tokens). A decoder-only (GPT-style) model would share the same context window for both, forcing the summary generation to compete with article storage. The encoder-decoder design instead compresses the source into a fixed-depth contextual representation via cross-attention, keeping source and target lengths fully decoupled.

2. BPE Tokenization (8K vocab) over Word-Level

News articles contain a long-tail of named entities (person names, geopolitical entities, product names) that are unseen at test time. A word-level tokenizer maps all unknowns to a single <unk> token, collapsing entity identity. BPE decomposes any unknown word into its constituent subword units, providing graceful degradation to character-level representation while keeping the vocabulary 12.5× smaller than word-level (8K vs ~100K).

3. Inverted-Pyramid Truncation (60/30/10 split)

Naïve tail-truncation of long articles discards conclusions and supporting evidence. Journalistic articles follow an inverted-pyramid writing convention: the most newsworthy facts (who, what, when, where) appear in the lead paragraph. The 60% head + 30% sampled middle + 10% tail

strategy preserves lead content, samples representative body content, and retains the article's closing context — all within the fixed encoder context window.

4. Dynamic Padding over Global Padding

Global padding pads every sequence to the dataset maximum length (~2000 tokens for CNN/DailyMail), wasting $O(n^2)$ attention compute on padding positions. Dynamic padding (`collate_batch`) pads only to the longest sequence within each mini-batch, reducing average sequence length by 30–50% and proportionally reducing memory and FLOPs per batch.

5. Label Smoothing ($\epsilon = 0.1$)

Without label smoothing, the cross-entropy loss drives logits toward $\pm\infty$ as the model becomes overconfident. Overconfident models reproduce high-frequency training phrases regardless of source content (hallucination). Label smoothing with $\epsilon = 0.1$ replaces the one-hot target with a soft distribution that penalises maximum-confidence predictions, acting as an output regulariser and improving calibration.

6. Two Independent Masks

Two fundamentally different masking problems exist in this architecture:

- **Padding mask:** eliminates spurious attention to semantically empty `<pad>` tokens (applied in both encoder self-attention and decoder cross-attention).
- **Causal mask:** prevents decoder position t from attending to future positions $t+1 \dots T$ during training with teacher forcing — ensuring training and inference are behaviourally consistent.

These are kept separate and combined via element-wise AND in the decoder self-attention layer.

Hyperparameter Configuration

Hyperparameter	Demo Value	Full-Scale Value	Rationale
<code>hidden_dim</code>	128	512	Embedding and attention dimensionality
<code>num_heads</code>	4	8	Number of independent attention heads
<code>num_layers</code>	2	6	Encoder and decoder stack depth
<code>ffn_dim</code>	256	2048	Feed-forward intermediate dimension (4× hidden)
<code>vocab_size</code>	500	8000	BPE vocabulary size
<code>src_max_len</code>	512	512	Max encoder input

Hyperparameter	Demo Value	Full-Scale Value	Rationale
<code>tgt_max_len</code>	100	100	tokens Max decoder output tokens
<code>dropout</code>	0.1	0.1	Dropout rate on attention weights and FFN
<code>label_smoothing</code>	0.1	0.1	Smoothing coefficient ϵ
<code>learning_rate</code>	0.001	0.0001 + warmup	Adam LR (full training uses schedule)
<code>batch_size</code>	1	32	Mini-batch size
<code>grad_clip_norm</code>	1.0	1.0	Gradient clipping threshold

Set up Hugging Face

```

from google.colab import userdata

# Get the Hugging Face token from Colab secrets
hf_token = userdata.get('HF_TOKEN')

if hf_token:
    print("Hugging Face token loaded successfully.")
else:
    print("Warning: Hugging Face token not found in Colab secrets.
    Proceeding without authentication, which might cause issues.")

Hugging Face token loaded successfully.

# Install required packages
import subprocess
import sys

packages = [
    'torch',          # PyTorch deep learning framework
    'numpy',          # Numerical computing
    'datasets',       # HuggingFace datasets for CNN/DailyMail
    'tokenizers',     # BPE tokenization library
    'rouge-score',    # ROUGE metrics evaluation
]

print("Installing required packages...")
for package in packages:
    try:
        __import__(package.replace('-', '_'))
        print(f"    {package}: Already installed")
    except ImportError:

```

```

        print(f"  Installing {package}...")
        subprocess.check_call([sys.executable, '-m', 'pip', 'install',
                              '-q', package])
        print(f"  {package}: Installed successfully")

print("All packages ready!\n")

# Import all required libraries
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import (
    Dataset,
    DataLoader
)
import math
from typing import (
    List,
    Dict,
    Tuple,
    Optional
)
from collections import defaultdict
import re
import html
import unicodedata
import numpy as np

# Setup device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"GPU Memory:
{torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

Installing required packages...
torch: Already installed
numpy: Already installed
datasets: Already installed
tokenizers: Already installed
Installing rouge-score...
rouge-score: Installed successfully
All packages ready!

Using device: cuda
GPU: Tesla T4
GPU Memory: 15.64 GB

```

Load CNN/DailyMail Dataset

```
from datasets import load_dataset
from google.colab import userdata

# Get the Hugging Face token from Colab secrets
hf_token = userdata.get('HF_TOKEN')

# This will automatically download and prepare the dataset
print("Downloading/Loading CNN/DailyMail dataset...")
dataset = load_dataset("abisee/cnn_dailymail", "3.0.0",
token=hf_token)

# Access the training, validation, and test splits
train_data = dataset['train']
val_data = dataset['validation']
test_data = dataset['test']

# Example of how to extract lists of articles and summaries for your
existing SummarizationDataset class
train_articles = train_data['article'][:1000] # Grabbing a subset to
test
train_summaries = train_data['highlights'][:1000]

print(f"Loaded {len(train_articles)} articles for training.")
```

Downloading/Loading CNN/DailyMail dataset...

```
{"model_id": "d818eec40132431e948706bfb138e25b", "version_major": 2, "version_minor": 0}

{"model_id": "be613aelcafb455f9d6adc275e864ba8", "version_major": 2, "version_minor": 0}

{"model_id": "ddb3a1a21b2e437e81b6ee54c4eee1d5", "version_major": 2, "version_minor": 0}

{"model_id": "5d8299cf4ab54690a35d9b1e62f54f60", "version_major": 2, "version_minor": 0}

{"model_id": "a4cc86f483b540208c01207cc0670bb1", "version_major": 2, "version_minor": 0}

{"model_id": "c4269d218a90497597d73742dc157dc6", "version_major": 2, "version_minor": 0}

{"model_id": "1173be03a5a34ad4a442db1c40ecc766", "version_major": 2, "version_minor": 0}

{"model_id": "ac56016abbb24fd6ad24a06d4676c709", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "lced06b8c1c14a1a8902dfab289ea7c8", "version_major": 2, "version_minor": 0}
```

Loaded 1000 articles for training.

```
from datasets import load_dataset
```

```
# This will automatically download and prepare the dataset  
print("Downloading/Loading CNN/DailyMail dataset...")
```

```
# Using the hf_token for authentication  
dataset = load_dataset("abisee/cnn_dailymail", "3.0.0",  
token=hf_token)
```

```
# Access the training, validation, and test splits  
train_data = dataset['train']  
val_data = dataset['validation']  
test_data = dataset['test']
```

```
# Example of how to extract lists of articles and summaries for your  
existing SummarizationDataset class  
train_articles = train_data['article'][:1000] # Grabbing a subset to  
test  
train_summaries = train_data['highlights'][:1000]
```

```
print(f"Loaded {len(train_articles)} articles for training.")
```

Downloading/Loading CNN/DailyMail dataset...
Loaded 1000 articles for training.

Verify dataset Structure

```
print(f"Type of dataset object: {type(dataset)}")  
print(f"Available splits in the dataset: {dataset.keys()}")
```

```
# Inspect the 'train' split  
print("\n--- Training Split (train_data) ---")  
print(f"Number of training examples: {len(train_data)}")  
print(f"Features (columns) in training data: {train_data.features}")  
print("First training example:")  
print(train_data[0])
```

```
# Inspect the 'validation' split  
print("\n--- Validation Split (val_data) ---")  
print(f"Number of validation examples: {len(val_data)}")  
print(f"Features (columns) in validation data: {val_data.features}")
```

```
# Inspect the 'test' split  
print("\n--- Test Split (test_data) ---")  
print(f"Number of test examples: {len(test_data)}")  
print(f"Features (columns) in test data: {test_data.features}")
```

```
Type of dataset object: <class 'datasets.dataset_dict.DatasetDict'>
Available splits in the dataset: dict_keys(['train', 'validation',
'test'])
```

```
--- Training Split (train_data) ---
```

```
Number of training examples: 287113
```

```
Features (columns) in training data: {'article': Value('string'),
'highlights': Value('string'), 'id': Value('string')}
```

```
First training example:
```

```
{'article': 'LONDON, England (Reuters) -- Harry Potter star Daniel
Radcliffe gains access to a reported £20 million ($41.1 million)
fortune as he turns 18 on Monday, but he insists the money won\'t cast
a spell on him. Daniel Radcliffe as Harry Potter in "Harry Potter and
the Order of the Phoenix" To the disappointment of gossip columnists
around the world, the young actor says he has no plans to fritter his
cash away on fast cars, drink and celebrity parties. "I don\'t plan to
be one of those people who, as soon as they turn 18, suddenly buy
themselves a massive sports car collection or something similar," he
told an Australian interviewer earlier this month. "I don\'t think
I\'ll be particularly extravagant. "The things I like buying are
things that cost about 10 pounds -- books and CDs and DVDs." At 18,
Radcliffe will be able to gamble in a casino, buy a drink in a pub or
see the horror film "Hostel: Part II," currently six places below his
number one movie on the UK box office chart. Details of how he\'ll
mark his landmark birthday are under wraps. His agent and publicist
had no comment on his plans. "I\'ll definitely have some sort of
party," he said in an interview. "Hopefully none of you will be
reading about it." Radcliffe\'s earnings from the first five Potter
films have been held in a trust fund which he has not been able to
touch. Despite his growing fame and riches, the actor says he is
keeping his feet firmly on the ground. "People are always looking to
say \'kid star goes off the rails,\'" he told reporters last month.
"But I try very hard not to go that way because it would be too easy
for them." His latest outing as the boy wizard in "Harry Potter and
the Order of the Phoenix" is breaking records on both sides of the
Atlantic and he will reprise the role in the last two films. Watch I-
Reporter give her review of Potter\'s latest » . There is life beyond
Potter, however. The Londoner has filmed a TV movie called "My Boy
Jack," about author Rudyard Kipling and his son, due for release later
this year. He will also appear in "December Boys," an Australian film
about four boys who escape an orphanage. Earlier this year, he made
his stage debut playing a tortured teenager in Peter Shaffer\'s
"Equus." Meanwhile, he is braced for even closer media scrutiny now
that he\'s legally an adult: "I just think I\'m going to be more sort
of fair game," he told Reuters. E-mail to a friend . Copyright 2007
Reuters. All rights reserved.This material may not be published,
broadcast, rewritten, or redistributed.', 'highlights': "Harry Potter
star Daniel Radcliffe gets £20M fortune as he turns 18 Monday .\nYoung
actor says he has no plans to fritter his cash away .\nRadcliffe's
earnings from first five Potter films have been held in trust fund .",
```

```
'id': '42c027e4ff9730fbb3de84c1af0d2c506e41c3e4'}

--- Validation Split (val_data) ---
Number of validation examples: 13368
Features (columns) in validation data: {'article': Value('string'),
'highlights': Value('string'), 'id': Value('string')}

--- Test Split (test_data) ---
Number of test examples: 11490
Features (columns) in test data: {'article': Value('string'),
'highlights': Value('string'), 'id': Value('string')}
```

MODULE 1: Document Processing & Linearization

Task 1: Content Truncation & Cleaning (1.5 Marks)

Objective

Perform text normalization by filtering noise, non-ASCII characters, and formatting anomalies. Implement a strict context-window truncation strategy to handle articles exceeding the Transformer's max sequence length while minimizing information loss.

Implementation Requirements

1. Text normalization (remove HTML, non-ASCII, whitespace)
 2. Metadata filtering (timestamps, author names, category tags)
 3. Context-window aware truncation (preserve important information)
 4. Handle edge cases (very short/long articles)
-

Conceptual Explanation

1. Why Text Cleaning Is a Prerequisite for Neural Text Processing

Raw web-scraped news articles carry significant non-semantic noise: HTML entity codes (& ,), XML/HTML tags from CMS templates, author by-lines, timestamps, and section labels injected by the publishing pipeline. If these artefacts are fed directly to a tokenizer, they consume context-window budget with zero informational value and introduce distribution shift — the model sees tokens like `<p>` or `January 15, 2023` during training that will never appear in clean inference input, degrading generalisation.

Unicode normalisation is equally critical for subword tokenizers: the byte sequence for `é` (U+00E9) differs from `e` + combining acute (U+0065 + U+0301). Without normalisation,

morphologically identical words may hash to different vocabulary entries, fragmenting learned representations.

2. The Context-Window Constraint and Information Loss

Transformer self-attention has $O(n^2)$ complexity in sequence length n . Practical deployments cap n at 512–2048 tokens. CNN/DailyMail articles average ~800 tokens; naïve tail-truncation would discard conclusions and supporting evidence that often appear mid-article.

The **inverted-pyramid truncation strategy** implemented here is grounded in journalistic writing conventions: the most newsworthy information (who, what, when, where) appears in the lead paragraph, with elaboration and background following. Preserving 60 % from the article head, sampling the middle 30 %, and retaining the final 10 % captures lead facts, representative body content, and concluding context while respecting the fixed budget.

Formally, let $W = \{w_1, w_2, \dots, w_N\}$ be the word sequence of length $N > L$ (the max length). The truncation function T selects indices:

```
T(W, L) = W[0 : 0.6L] ∪ W[0.6L : N-0.1L : stride] ∪ W[N-0.1L : N]
where stride = [(N - 0.7L) / 0.3L]
```

This yields exactly L words with maximum distributional coverage across the original article.

3. Encoder-Decoder Advantage for Asymmetric Length Tasks

A decoder-only model (e.g., GPT) processes source and target as a single concatenated sequence. For summarization, the source article (512 tokens) and target summary (50 tokens) together consume 562 tokens of the context window, and the causal mask forces each summary token to attend over all 512 source tokens individually through the same attention mechanism used for language modelling.

An encoder-decoder architecture instead **compresses the source into a fixed-depth contextual representation** in the encoder stack. The decoder attends to this representation via cross-attention, not to the raw token sequence. This provides three advantages:

- Source and target sequence lengths are decoupled — the encoder can process 1024-token articles while the decoder generates 50-token summaries without combined context overflow.
- The encoder applies bidirectional (non-causal) attention, allowing every source token to incorporate context from both past and future tokens — critical for resolving coreference ("He said" → who?) before the decoder begins generation.
- Computation scales with $\max(N_{\text{src}}, N_{\text{tgt}})$ rather than $N_{\text{src}} + N_{\text{tgt}}$, which is materially cheaper for highly asymmetric length ratios such as those in summarization.

```
class TextCleaner:
    """
    Clean raw text from web sources.

    Removes:
    - HTML entities and tags
    - Non-ASCII characters
```

```

- Extra whitespace
- Metadata (timestamps, author names)
"""

def __init__(self, remove_html=True, remove_non_ascii=True):
    self.remove_html = remove_html
    self.remove_non_ascii = remove_non_ascii

def clean(self, text):
    # Step 1: HTML entity decoding
    if self.remove_html:
        text = html.unescape(text)

    # Step 2: Remove HTML/XML tags
    text = re.sub(r'<[^>]+>', '', text)

    # Step 3: Keep only ASCII characters
    if self.remove_non_ascii:
        text = ''.join(c for c in text if ord(c) < 128)

    # Step 4: Normalize whitespace
    text = re.sub(r'\s+', ' ', text)

    # Step 5: Remove timestamps
    text = re.sub(r'\d{1,2}/\d{1,2}/\d{4}', '', text)
    text = re.sub(r'(January|February|March|April|May|June|July|
August|September|October|November|December)\s+\d{1,2},?\s+\d{4}', '',
text)

    # Step 6: Remove bylines
    text = re.sub(r'(?i)by\s+[A-Z][a-z]+\s+[A-Z][a-z]+', '', text)
    text = re.sub(r'[A-Z\s]+--', '', text)

    return text.strip()

class DocumentTruncator:
    """
    Intelligently truncate documents to fit within context window.

    Strategy: Preserve first 60% (salient info), sample middle 30%,
    keep last 10%
    This follows journalistic "inverted pyramid" style where important
    info appears first.
    """

    def __init__(self, max_length=512):
        self.max_length = max_length

    def truncate(self, text):

```

```

words = text.split()

if len(words) <= self.max_length:
    return text

# Calculate section sizes
first_len = int(self.max_length * 0.6)      # 60% for leading
info
last_len = int(self.max_length * 0.1)      # 10% for
conclusion
middle_len = self.max_length - first_len - last_len # 30%
sampling from middle

# Extract sections
first_section = words[:first_len]
last_section = words[-last_len:]

# Sample middle section with stride
middle_words = words[first_len:-last_len]
stride = max(1, len(middle_words) // middle_len)
middle_section = middle_words[::stride][:middle_len]

# Combine sections
result = first_section + middle_section + last_section
return ' '.join(result)

# Test Task 1
cleaner = TextCleaner()
truncator = DocumentTruncator(max_length=50)

test_text = "<p>The & quick&nbsp;<b>brown</b> fox jumps. Published
January 15, 2023 by John Smith. " + " ".join([f"word{i}" for i in
range(100)]) + "</p>"

cleaned = cleaner.clean(test_text)
truncated = truncator.truncate(cleaned)

print("TASK 1: Content Truncation & Cleaning")
print(f"Original length: {len(test_text.split())} words")
print(f"After cleaning: {len(cleaned.split())} words")
print(f"After truncation: {len(truncated.split())} words")
print(f"Cleaned text (first 100 chars): {cleaned[:100]}...")

TASK 1: Content Truncation & Cleaning
Original length: 112 words
After cleaning: 107 words
After truncation: 50 words
Cleaned text (first 100 chars): The & quickbrown fox jumps. Published
. word0 word1 word2 word3 word4 word5 word6 word7 word8 word9...

```

Task 2: Subword Tokenization & Dynamic Padding (1.5 Marks)

Objective

Train a custom subword tokenizer (BPE/WordPiece) on your summarization corpus. Provide a technical justification explaining why subword tokenization is essential for handling out-of-vocabulary (OOV) names, places, or concepts frequently found in news datasets.

Technical Justification

Why Subword Tokenization?

News articles contain highly diverse, unpredictable vocabulary:

- Named entities: "Johannesburg", "Emmanuel Macron", "Tesla"
- Technical terms: "cryptocurrency", "algorithmic bias"
- Rare / neologism words: "serendipity", "COVID-19"

Problem with Word-Level Tokenization (100K vocab):

- Every unique entity occupies its own embedding row → sparse representation
- Any word not seen at training time → `<unk>` → information loss
- Enormous embedding matrices → memory and speed cost
- Cannot generalise morphological variants: `run` ≠ `running` ≠ `runner`

BPE (Byte-Pair Encoding) Solution (8K vocab):

BPE starts with a character-level vocabulary and iteratively merges the most-frequent adjacent symbol pair:

```
Step 0 (chars): J o h a n n e s b u r g </w>
Step 1 (merge 'n', 'n'): J o h a n n e s b u r g </w>
Step 2 (merge 'J', 'o'): J o h a n n e s b u r g </w>
...                'Johan' + 'nes' + 'burg</w>'
```

- Shared subword units across related words: `Johan` also seen in `John`, `Johnson`
- Any OOV word decomposes to characters at worst — **no `<unk>` for real text**
- 12.5× smaller vocabulary → 12.5× smaller embedding matrix
- Morphological patterns (`-ing`, `-tion`, `un-`) emerge automatically from frequency statistics

Why BPE over WordPiece here: BPE merges are based on raw co-occurrence frequency and require no external language model, making it straightforward to train from scratch on any corpus without additional dependencies.

Implementation Notes

- `BPETokenizer.train(corpus)` — runs the full BPE training loop on a list of strings

- `encode(text) / decode(ids)` — identical interface to the rest of the pipeline
- `SummarizationDataset` and `collate_batch` are **unchanged** — they consume any tokenizer with `encode/decode/get_vocab_size`
- **When using CNN/DailyMail:** call `tokenizer.train(train_articles + train_summaries)` before constructing the dataset

Conceptual Explanation

1. The OOV Problem in Word-Level Tokenization

A word-level tokenizer with vocabulary V maps every training-set word to a unique integer. At inference time any word $w \notin V$ is mapped to a single `<unk>` token regardless of its morphological similarity to known words. For news text this is severe: named entities (person names, geopolitical entities, product names) have a long-tail frequency distribution — the majority appear fewer than five times in even a 287 K-document corpus. Mapping all of them to `<unk>` collapses the model's ability to propagate entity identity through the summary.

2. BPE Algorithm — Formal Description

Sennrich et al. (2016) adapt the data-compression BPE algorithm for NLP. The algorithm operates on a character-level initial vocabulary augmented with a word-boundary marker `</w>`:

```

Input : corpus C, target vocabulary size V
Output: merge table M, token vocabulary  $\Sigma$ 

1.  $\Sigma \leftarrow \{ c \text{ </w>} : c \in \text{characters}(C) \} \cup \text{special tokens}$ 
2. freq_vocab  $\leftarrow$  character-frequency word dict from C
3. while  $|\Sigma| < V$ :
    (a, b)  $\leftarrow \text{argmax}_{\{(x,y)\}} \text{count}(x \ y \text{ in } \text{freq\_vocab})$ 
    M  $\leftarrow M \cup \{ (a, b) \rightarrow ab \}$ 
    freq_vocab  $\leftarrow \text{apply\_merge}((a, b), \text{freq\_vocab})$ 
     $\Sigma \leftarrow \Sigma \cup \{ ab \}$ 
4. return M,  $\Sigma$ 

```

At encode time, the same merge rules are applied in training order to decompose any input word into the longest-matching subword units present in Σ .

3. Why Subword Units Solve OOV for News

Scenario	Word-level	BPE (8K vocab)
"Johannesburg" unseen in training	$\rightarrow$ <code><unk></code>	$\rightarrow$ <code>Johan + nes + burg</w></code>
"COVID-19"	$\rightarrow$ <code><unk></code>	$\rightarrow$ <code>C + 0 + V + ID + - + 19</w></code>
"transforming"	1 token (if seen)	$\rightarrow$ <code>transform + ing</w></code> (shares <code>transform</code> with <code>transforms</code> , <code>transformed</code>)

Scenario	Word-level	BPE (8K vocab)
Vocabulary size	~100K	8K
Embedding matrix (d=512)	51.2M params	4.1M params

The key insight is that subword decomposition provides **graceful degradation**: even a fully unknown word produces a sequence of character-level tokens that carry phonological and orthographic signal, rather than a single opaque `<unk>`.

4. Dynamic Padding — Why Batch-Level Is Preferred Over Global

Global padding (pad all sequences in the dataset to the maximum article length) wastes compute quadratically in the padded positions due to the $O(n^2)$ attention cost. Dynamic padding pads each mini-batch only to the longest sequence *in that batch*. For CNN/DailyMail where article lengths range from 50 to 2000 tokens, this typically reduces average sequence length by 30–50 %, translating directly to proportional reduction in memory and FLOPs per batch.

```
from collections import defaultdict
import re

class BPETokenizer:
    """
    Byte-Pair Encoding (BPE) Subword Tokenizer – trained from scratch.

    BPE Algorithm:
    1. Initialise vocabulary as individual characters of every word
       (end-of-word marked with '</w>' so the model can reconstruct
       spaces).
    2. Count every adjacent symbol pair across the entire corpus.
    3. Merge the most-frequent pair into a single new token.
    4. Repeat steps 2-3 until the target vocab_size is reached.

    Why BPE for news summarisation:
    - Rare named entities decompose gracefully:
      'Johannesburg' -> 'Johan', '##nes', '##burg'
      The model can reuse 'Johan' learned from 'John', 'Johnson', etc.
    - Technical / new terms ('cryptocurrency', 'COVID-19') never
      produce <unk>;
      they fall back to character-level tokens at worst.
    - Vocabulary stays compact (8K vs 100K+ word-level), so the
      embedding
      matrix is 12.5x smaller and generalises better across domains.
    - Morphological patterns ('-ing', '-tion', 'un-') emerge
      automatically
      from frequency statistics without any linguistic hand-crafting.
    """

    #
    ----- #
```

```

# Constructor
#
----- #

def __init__(self, vocab_size=8000):
    self.vocab_size = vocab_size
    self.special_tokens = {'<pad>': 0, '<start>': 1, '<end>': 2,
'<unk>': 3}
    self.token_to_id = dict(self.special_tokens)
    self.id_to_token = {v: k for k, v in
self.token_to_id.items()}
    self.merges = {} # (tokenA, tokenB) -> merged_token
    (ordered dict)
    self.trained = False
    self._next_id = len(self.special_tokens)

#
----- #

# Private helpers
#
----- #

def _word_to_chars(self, word):
    """Split word into characters; append </w> to the last char to
mark word boundary."""
    chars = list(word)
    chars[-1] = chars[-1] + '</w>'
    return chars

def _build_char_vocab(self, corpus):
    """
    Build initial character-level word-frequency dict from corpus.
    Each key is a tuple of characters; value is word frequency.
    """
    vocab = defaultdict(int)
    for text in corpus:
        for word in text.lower().split():
            key = tuple(self._word_to_chars(word))
            vocab[key] += 1
    return vocab

def _get_pair_frequencies(self, vocab):
    """Count how often every adjacent symbol pair appears across
the vocab."""
    pairs = defaultdict(int)
    for word_tuple, freq in vocab.items():
        for i in range(len(word_tuple) - 1):
            pairs[(word_tuple[i], word_tuple[i + 1])] += freq

```

```

        return pairs

    def _apply_merge(self, pair, vocab):
        """
        Apply one BPE merge rule to every entry in the vocabulary.
        Uses regex to replace 'A B' with 'AB' only at symbol
        boundaries.
        """
        merged = pair[0] + pair[1]
        bigram = re.escape(' '.join(pair))
        pattern = re.compile(r'(?<[\S])' + bigram + r'(?![\S])')
        new_vocab = {}
        for word_tuple, freq in vocab.items():
            word_str = ' '.join(word_tuple)
            new_word_str = pattern.sub(merged, word_str)
            new_vocab[tuple(new_word_str.split())] = freq
        return new_vocab

    def _register_token(self, token):
        """Add a token to the vocabulary if it is not already
        present."""
        if token not in self.token_to_id:
            self.token_to_id[token] = self._next_id
            self.id_to_token[self._next_id] = token
            self._next_id += 1

#
----- #
# Training
#
#
----- #

    def train(self, corpus):
        """
        Train BPE on a list of raw text strings.

        Steps:
        1. Build character-level vocabulary from corpus.
        2. Register all seed characters as tokens.
        3. Greedily merge the most-frequent adjacent pair until
            vocab_size is reached or no pairs remain.

        Args:
            corpus (List[str]): Training sentences / documents.
        """
        vocab = self._build_char_vocab(corpus)

        # Seed: register every initial character token
        for word_tuple in vocab:

```

```

        for ch in word_tuple:
            self._register_token(ch)

    num_merges = self.vocab_size - self._next_id
    for step in range(max(0, num_merges)):
        pairs = self._get_pair_frequencies(vocab)
        if not pairs:
            break # corpus fully merged
        best_pair = max(pairs, key=pairs.get)
        vocab = self._apply_merge(best_pair, vocab)
        merged_token = best_pair[0] + best_pair[1]
        self.merges[best_pair] = merged_token
        self._register_token(merged_token)

    self.trained = True
    print(f" BPE training complete: {len(self.merges)} merge
rules, "
        f"vocab size = {self._next_id}")

#
----- #
# Encoding / Decoding
#
#
----- #

def _apply_merges_to_word(self, word):
    """
    Apply all learned BPE merges (in training order) to one word.
    Returns a list of subword tokens.
    """
    symbols = self._word_to_chars(word)
    for pair, merged in self.merges.items():
        i = 0
        while i < len(symbols) - 1:
            if (symbols[i], symbols[i + 1]) == pair:
                symbols = symbols[:i] + [merged] + symbols[i + 2:]
                # Don't advance i: newly merged token may
                participate again
            else:
                i += 1
        return symbols

def encode(self, text):
    """
    Encode a string into a list of integer token IDs.
    Falls back to <unk> (id=3) for any token not seen during
    training.
    """
    ids = []

```

```

        for word in text.lower().split():
            for subword in self._apply_merges_to_word(word):
                ids.append(
                    self.token_to_id.get(subword,
self.special_tokens['<unk>'])
                )
            return ids

    def decode(self, ids):
        """
        Decode a list of token IDs back into a human-readable string.
        Strips </w> end-of-word markers and restores word spacing.
        """
        tokens = [self.id_to_token.get(i, '<unk>') for i in ids]
        text = ' '.join(tokens)
        # Merge '</w>' boundaries back into natural word spacing
        text = text.replace('</w> ', ' ').replace('</w>', '')
        return text.strip()

    def get_vocab_size(self):
        """Return the current number of tokens in the vocabulary."""
        return self._next_id

#
=====
= #
# SummarizationDataset & collate_batch – unchanged from Task 1
interface #
#
=====
= #

class SummarizationDataset(Dataset):
    """PyTorch Dataset for (article, summary) pairs with dynamic
padding."""

    def __init__(self, articles, summaries, tokenizer,
                 src_max_len=512, tgt_max_len=100):
        self.articles = articles
        self.summaries = summaries
        self.tokenizer = tokenizer
        self.src_max_len = src_max_len
        self.tgt_max_len = tgt_max_len

    def __len__(self):
        return len(self.articles)

    def __getitem__(self, idx):
        article = self.articles[idx]

```

```

summary = self.summaries[idx]

# Tokenize with real BPE subword units
src_ids = self.tokenizer.encode(article)[:self.src_max_len]
tgt_ids = self.tokenizer.encode(summary)[:self.tgt_max_len]

# Wrap target with <start> (1) and <end> (2) special tokens
tgt_ids = [1] + tgt_ids + [2]

return {
    'src': torch.tensor(src_ids, dtype=torch.long),
    'tgt': torch.tensor(tgt_ids, dtype=torch.long),
    'src_len': len(src_ids),
    'tgt_len': len(tgt_ids),
}

def collate_batch(batch):
    """
    Dynamic padding collate function.

    Pads each mini-batch only to *its own* maximum sequence length
rather
than to a global maximum. This saves memory and compute,
especially
important with BPE where article lengths vary significantly.
    """
    max_src_len = max(item['src_len'] for item in batch)
    max_tgt_len = max(item['tgt_len'] for item in batch)

    src_batch = []
    tgt_batch = []
    src_mask_batch = []

    for item in batch:
        src = item['src']
        src_padded = F.pad(src, (0, max_src_len - len(src)), value=0)
# pad_id = 0
        src_batch.append(src_padded)

        # Attention mask: 1 = real token, 0 = padding
        src_mask = torch.ones(max_src_len)
        src_mask[len(src):] = 0
        src_mask_batch.append(src_mask)

        tgt = item['tgt']
        tgt_padded = F.pad(tgt, (0, max_tgt_len - len(tgt)), value=0)
        tgt_batch.append(tgt_padded)

    return {

```

```

        'src':      torch.stack(src_batch),
        'tgt':      torch.stack(tgt_batch),
        'src_mask': torch.stack(src_mask_batch),
    }

#
=====
= #
# Task 2 Test & Demonstration
#
#
=====
= #

print("TASK 2: Subword BPE Tokenization & Dynamic Padding")
print("-" * 55)

# Sample corpus used to train the BPE tokenizer
articles = [
    "the quick brown fox jumps over the lazy dog in the forest",
    "artificial intelligence is transforming the world",
]
summaries = [
    "fox jumps over dog",
    "ai transforms world",
]

# 1. Instantiate and TRAIN the BPE tokenizer on the corpus
tokenizer = BPETokenizer(vocab_size=500)
print("Training BPE tokenizer on corpus...")
tokenizer.train(articles + summaries)

# 2. Demonstrate real subword decomposition
print("\nSubword decomposition examples:")
for word in ["transforming", "artificial", "johannesburg"]:
    subwords = tokenizer._apply_merges_to_word(word)
    print(f"    '{word}' -> {subwords}")

# 3. Build dataset and DataLoader (unchanged interface)
dataset = SummarizationDataset(articles, summaries, tokenizer)
loader = DataLoader(dataset, batch_size=2, collate_fn=collate_batch)

batch = next(iter(loader))
print(f"\nTokenizer vocabulary size : {tokenizer.get_vocab_size()}")
print(f"Source batch shape           : {batch['src'].shape}")
print(f"Target batch shape           : {batch['tgt'].shape}")
print(f"Source mask shape             : {batch['src_mask'].shape}")
print(f"Merge rules learned          : {len(tokenizer.merges)}")
print("\nNote: When running on CNN/DailyMail, call

```

```
tokenizer.train(train_articles + train_summaries)")
print("        before constructing SummarizationDataset so all corpus
tokens are in the vocabulary.")
```

TASK 2: Subword BPE Tokenization & Dynamic Padding

Training BPE tokenizer on corpus...
BPE training complete: 63 merge rules, vocab size = 102

Subword decomposition examples:

```
'transforming' -> ['transforming</w>']
'artificial' -> ['artificial</w>']
'johannesburg' -> ['j', 'o', 'h', 'a', 'n', 'n', 'e', 's', 'b', 'u',
'r', 'g</w>']
```

```
Tokenizer vocabulary size : 102
Source batch shape       : torch.Size([2, 12])
Target batch shape       : torch.Size([2, 6])
Source mask shape        : torch.Size([2, 12])
Merge rules learned      : 63
```

Note: When running on CNN/DailyMail, call
tokenizer.train(train_articles + train_summaries)
before constructing SummarizationDataset so all corpus tokens
are in the vocabulary.

MODULE 2: Structural Architecture & Information Bottlenecks

Task 3: Encoder-Decoder & Cross-Attention Block Construction (2 Marks)

Objective

Implement the Multi-Head Self-Attention layers for both the Encoder and Decoder blocks from scratch. Build the Cross-Attention layer to map the Decoder's target sequence queries directly to the Encoder's source article contextual keys and values.

Key Components

1. Multi-Head Self-Attention for encoder
2. Multi-Head Self-Attention (causal) for decoder
3. Cross-Attention for decoder to encoder binding
4. Feed-forward networks

Conceptual Explanation

1. Scaled Dot-Product Attention — Mathematical Foundation

The attention function maps a query matrix Q , key matrix K , and value matrix V to an output:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

where d_k is the key dimension. The scaling factor $1/\sqrt{d_k}$ prevents the dot products from growing large in magnitude as d_k increases, which would push the softmax into saturation regions with near-zero gradients. Intuitively, each query vector *asks* which keys are most relevant; the softmax produces a probability distribution over positions, and the output is a weighted sum of values.

2. Multi-Head Attention — Representational Diversity

Rather than computing a single attention function over d_{model} -dimensional vectors, multi-head attention projects Q, K, V into h independent $d_k = d_{\text{model}}/h$ -dimensional subspaces:

$$\begin{aligned} \text{head}_i &= \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V) \\ \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \end{aligned}$$

Each head learns to attend to a different relational pattern simultaneously. In summarization, empirical analysis shows heads specialise: some attend to subject-verb pairs (preserving predicate structure in summaries), others to coreference chains (linking pronouns to named entities), and others to positional proximity. Concatenating all heads and projecting back to d_{model} fuses these complementary perspectives into a single enriched representation.

3. Cross-Attention — The Information Bottleneck

Cross-attention is the architectural mechanism that makes encoder-decoder models suitable for transduction tasks. The decoder's current hidden state provides the **queries** (what information does the decoder need next?), while the encoder's output provides the **keys and values** (what information is available in the source?):

$$\text{CrossAttention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{enc}}^T}{\sqrt{d_k}}\right) \cdot V_{\text{enc}}$$

This creates a soft, differentiable alignment: each generated summary token can selectively attend to whichever source article tokens are most relevant at that generation step — analogous to how a human writer scans back through source material while composing a sentence.

Critically, the **keys and values are fixed** after the encoder pass and are **shared across all decoder layers and all decoding timesteps**. This is the information bottleneck: the decoder cannot access the raw article tokens directly; it can only retrieve information through the

compressed contextual representations produced by the encoder, forcing the model to learn to produce good abstractions in the encoder rather than copying verbatim.

4. Residual Connections and Layer Normalisation

Each sub-layer (attention, FFN) is wrapped with a residual connection and layer normalisation:

```
x ← LayerNorm( x + SubLayer(x) )
```

Residual connections address the vanishing gradient problem in deep stacks by providing gradient highways that bypass sub-layers. Layer normalisation stabilises the activation distribution across the feature dimension (not the batch dimension), making training insensitive to batch size — important when articles in a batch have very different lengths and therefore very different activation magnitudes.

5. Why Encoder-Decoder Outperforms Decoder-Only for Summarization

Property	Decoder-only (GPT-style)	Encoder-Decoder (T5/BART-style)
Source attention	Causal — each token attends only to past	Bidirectional — full article context per token
Context budget	Shared between source + target	Separate; source length does not consume target budget
Length asymmetry	Inefficient for long-src/short-tgt	Native: encoder compresses, decoder expands
Coreference resolution	Partial (left-context only)	Full (bidirectional encoder)
Empirical ROUGE on CNN/DM	Lower (GPT-2: R1 ≈ 29)	Higher (BART: R1 ≈ 44)

```
class MultiHeadAttention(nn.Module):
    """
    Multi-Head Self-Attention mechanism.

    Multiple heads allow attending to different representation
    subspaces:
    - Head 1: Factual entities
    - Head 2: Causal relationships
    - Head 3: Temporal information
    - etc.
    """

    def __init__(self, hidden_dim, num_heads=8, dropout=0.1):
        super().__init__()
        assert hidden_dim % num_heads == 0

        self.hidden_dim = hidden_dim
        self.num_heads = num_heads
        self.head_dim = hidden_dim // num_heads
```

```

    # Linear projections
    self.linear_q = nn.Linear(hidden_dim, hidden_dim)
    self.linear_k = nn.Linear(hidden_dim, hidden_dim)
    self.linear_v = nn.Linear(hidden_dim, hidden_dim)
    self.linear_out = nn.Linear(hidden_dim, hidden_dim)

    self.dropout = nn.Dropout(dropout)

def forward(self, query, key, value, mask=None):
    batch_size = query.size(0)

    # Project Q, K, V
    Q = self.linear_q(query)
    K = self.linear_k(key)
    V = self.linear_v(value)

    # Reshape for multi-head
    Q = Q.view(batch_size, -1, self.num_heads,
self.head_dim).transpose(1, 2)
    K = K.view(batch_size, -1, self.num_heads,
self.head_dim).transpose(1, 2)
    V = V.view(batch_size, -1, self.num_heads,
self.head_dim).transpose(1, 2)

    # Scaled dot-product attention
    scores = torch.matmul(Q, K.transpose(-2, -1)) /
math.sqrt(self.head_dim)

    # Apply mask
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # Attention weights
    weights = F.softmax(scores, dim=-1)
    weights = self.dropout(weights)

    # Apply to values
    context = torch.matmul(weights, V)

    # Reshape back
    context = context.transpose(1, 2).contiguous()
    context = context.view(batch_size, -1, self.hidden_dim)

    # Output projection
    output = self.linear_out(context)

    return output, weights

class PositionalEncoding(nn.Module):

```

```

"""Sinusoidal positional encoding."""

def __init__(self, hidden_dim, max_len=5000, dropout=0.1):
    super().__init__()
    self.dropout = nn.Dropout(dropout)

    # Create positional encodings
    pe = torch.zeros(max_len, hidden_dim)
    position = torch.arange(0, max_len).unsqueeze(1).float()
    div_term = torch.exp(torch.arange(0, hidden_dim, 2).float() *
                          -(math.log(10000.0) / hidden_dim))

    pe[:, 0::2] = torch.sin(position * div_term)
    if hidden_dim % 2 == 1:
        pe[:, 1::2] = torch.cos(position * div_term[:-1])
    else:
        pe[:, 1::2] = torch.cos(position * div_term)

    self.register_buffer('pe', pe.unsqueeze(0))

def forward(self, x):
    return self.pe[:, :x.size(1), :]

class FeedForward(nn.Module):
    """Position-wise feed-forward network."""

    def __init__(self, hidden_dim, ffn_dim=2048, dropout=0.1):
        super().__init__()
        self.linear1 = nn.Linear(hidden_dim, ffn_dim)
        self.linear2 = nn.Linear(ffn_dim, hidden_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        return self.linear2(self.dropout(F.relu(self.linear1(x))))

class EncoderBlock(nn.Module):
    """Encoder block with self-attention and feed-forward."""

    def __init__(self, hidden_dim, num_heads=8, ffn_dim=2048,
dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(hidden_dim, num_heads,
dropout)
        self.ffn = FeedForward(hidden_dim, ffn_dim, dropout)
        self.norm1 = nn.LayerNorm(hidden_dim)
        self.norm2 = nn.LayerNorm(hidden_dim)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)

```

```

def forward(self, x, mask=None):
    # Self-attention with residual
    attn_out, _ = self.self_attn(x, x, x, mask)
    x = x + self.dropout1(attn_out)
    x = self.norm1(x)

    # Feed-forward with residual
    ffn_out = self.ffn(x)
    x = x + self.dropout2(ffn_out)
    x = self.norm2(x)

    return x

class DecoderBlock(nn.Module):
    """Decoder block with self-attention, cross-attention, and feed-forward."""

    def __init__(self, hidden_dim, num_heads=8, ffn_dim=2048, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(hidden_dim, num_heads, dropout)
        self.cross_attn = MultiHeadAttention(hidden_dim, num_heads, dropout)
        self.ffn = FeedForward(hidden_dim, ffn_dim, dropout)
        self.norm1 = nn.LayerNorm(hidden_dim)
        self.norm2 = nn.LayerNorm(hidden_dim)
        self.norm3 = nn.LayerNorm(hidden_dim)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)
        self.dropout3 = nn.Dropout(dropout)

    def forward(self, x, encoder_output, self_mask=None, cross_mask=None):
        # Self-attention
        attn_out, _ = self.self_attn(x, x, x, self_mask)
        x = x + self.dropout1(attn_out)
        x = self.norm1(x)

        # Cross-attention to encoder
        cross_out, cross_weights = self.cross_attn(x, encoder_output, encoder_output, cross_mask)
        x = x + self.dropout2(cross_out)
        x = self.norm2(x)

        # Feed-forward
        ffn_out = self.ffn(x)
        x = x + self.dropout3(ffn_out)

```

```

        x = self.norm3(x)

        return x, cross_weights

print("TASK 3: Encoder-Decoder & Cross-Attention Blocks")
print("Multi-head attention: Implemented")
print("Positional encoding: Implemented")
print("Encoder block: Implemented")
print("Decoder block with cross-attention: Implemented")

TASK 3: Encoder-Decoder & Cross-Attention Blocks
Multi-head attention: Implemented
Positional encoding: Implemented
Encoder block: Implemented
Decoder block with cross-attention: Implemented

```

Task 4: Causal & Sequence Masking Implementation (1.5 Marks)

Objective

Design and deploy two distinct masks:

1. Absolute padding mask for source article (prevent attention on pad tokens)
 2. Causal look-ahead mask for summary generation (ensure proper autoregressive generation)
-

Conceptual Explanation

1. Padding Mask — Eliminating Spurious Attention

Dynamic batching requires sequences of unequal length to be padded to a common length using a reserved `<pad>` token (id = 0). Without masking, attention scores are computed between real tokens and pad positions, and softmax normalises over *all* positions including padding. This has two harmful effects:

1. **Information corruption:** real token representations are contaminated by attending to semantically empty pad positions.
2. **Gradient leakage:** pad positions receive non-zero attention weight, creating spurious gradients that do not correspond to any meaningful linguistic relationship.

The padding mask sets the attention logit for all pad positions to $-\infty$ before softmax:

```

score[i, j] = score[i, j]  if token[j] ≠ <pad>
score[i, j] = -∞          if token[j] = <pad>

```

After softmax, $\exp(-\infty) = 0$, so pad positions receive exactly zero attention weight. Shape: `(batch, 1, 1, src_len)` — the singleton dimensions broadcast across all heads and all query positions.

2. Causal (Look-Ahead) Mask — Enforcing Autoregressive Validity

The decoder generates tokens sequentially: token t is conditioned on tokens $1 \dots t-1$. During *training* with teacher forcing, the entire target sequence is fed to the decoder in parallel (for efficiency). Without masking, position t would attend to positions $t+1, t+2, \dots$ — future tokens that do not exist at inference time. This constitutes **target leakage**: the model learns a trivially easy shortcut (copy the next token) that does not generalise to autoregressive inference.

The causal mask is a lower-triangular binary matrix:

```
M[i, j] = 1 if j ≤ i (position i may attend to position j)
M[i, j] = 0 if j > i (position i must not attend to future position j)
```

For a 4-token sequence:

	t1	t2	t3	t4
t1	[1	0	0	0]
t2	[1	1	0	0]
t3	[1	1	1	0]
t4	[1	1	1	1]

Shape: `(1, 1, tgt_len, tgt_len)` — broadcasts over batch and head dimensions.

3. Combined Decoder Mask

The decoder self-attention layer requires *both* constraints simultaneously: target positions must not attend to future tokens *and* must not attend to target padding. The combined mask is the element-wise AND:

```
tgt_pad_mask = create_padding_mask(tgt_input)      # (batch, 1, 1, tgt_len)
causal_mask   = create_causal_mask(tgt_len, device) # (1, 1, tgt_len, tgt_len)
decoder_mask  = tgt_pad_mask & causal_mask         # (batch, 1, tgt_len, tgt_len)
```

This ensures every decoder query sees only past non-padding positions — the exact condition that makes training and inference behaviourally consistent.

4. Why These Two Masks Are Architecturally Distinct

Property	Padding Mask	Causal Mask
Applied in	Encoder self-attn, Decoder cross-attn	Decoder self-attn only
Shape	<code>(batch, 1, 1,</code>	<code>(1, 1, seq_len,</code>

Property	Padding Mask	Causal Mask
	<code>seq_len</code>	<code>seq_len</code>
Varies across	Batch (different pad positions)	Fixed for a given sequence length
Purpose	Remove padding noise	Prevent future information leakage
Inference	Same mask (input padding)	Not needed (generate left-to-right, no future tokens exist)

```
def create_padding_mask(seq, pad_token=0):
    """
    Create padding mask to hide pad tokens.

    Returns 1 for real tokens, 0 for padding.
    Prevents attention from wasting computation on padding.
    """
    mask = (seq != pad_token).float()
    return mask.unsqueeze(1).unsqueeze(1) # Shape: (batch, 1, 1, seq_len)

def create_causal_mask(seq_len, device):
    """
    Create causal mask for autoregressive generation.

    Token i can only attend to positions 0...i (not future positions).
    Implemented as lower triangular matrix.
    """
    # Create the tensor on CPU first, then move to device to
    # potentially
    # avoid rare device-side assert issues with direct device
    # creation.
    mask = torch.ones(seq_len, seq_len)
    mask = mask.to(device)
    mask = torch.tril(mask)
    return mask.unsqueeze(0).unsqueeze(0) # Shape: (1, 1, seq_len, seq_len)

# Test Task 4
print("\nTASK 4: Causal & Sequence Masking")

# Test padding mask
seq = torch.tensor([[1, 2, 3, 0, 0]]) # Last two are padding
pad_mask = create_padding_mask(seq, pad_token=0)
print(f"Padding mask shape: {pad_mask.shape}")
print(f"Padding mask (1=attend, 0=ignore): {pad_mask[0, 0, 0]}")

# Test causal mask
```

```
causal = create_causal_mask(4, device)
print(f"\nCausal mask shape: {causal.shape}")
print(f"Causal mask (lower triangular):\n{causal[0, 0].int()}")
```

TASK 4: Causal & Sequence Masking

Padding mask shape: torch.Size([1, 1, 1, 5])

Padding mask (1=attend, 0=ignore): tensor([1., 1., 1., 0., 0.])

Causal mask shape: torch.Size([1, 1, 4, 4])

Causal mask (lower triangular):

```
tensor([[1, 0, 0, 0],
        [1, 1, 0, 0],
        [1, 1, 1, 0],
        [1, 1, 1, 1]], device='cuda:0', dtype=torch.int32)
```

Full Transformer Model

```
class Encoder(nn.Module):
    """Transformer encoder (bidirectional)."""

    def __init__(self, vocab_size, hidden_dim=512, num_heads=8,
                 num_layers=6, ffn_dim=2048, dropout=0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.pos_encoding = PositionalEncoding(hidden_dim,
        dropout=dropout)
        self.layers = nn.ModuleList(
            [EncoderBlock(hidden_dim, num_heads, ffn_dim, dropout)
             for _ in range(num_layers)]
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, mask=None):
        x = self.embedding(src) *
math.sqrt(self.embedding.embedding_dim)
        x = x + self.pos_encoding(x)
        x = self.dropout(x)

        for layer in self.layers:
            x = layer(x, mask)

        return x

class Decoder(nn.Module):
    """Transformer decoder (autoregressive)."""
```

```

def __init__(self, vocab_size, hidden_dim=512, num_heads=8,
              num_layers=6, ffn_dim=2048, dropout=0.1):
    super().__init__()
    self.embedding = nn.Embedding(vocab_size, hidden_dim)
    self.pos_encoding = PositionalEncoding(hidden_dim,
dropout=dropout)
    self.layers = nn.ModuleList(
        [DecoderBlock(hidden_dim, num_heads, ffn_dim, dropout)
         for _ in range(num_layers)]
    )
    self.dropout = nn.Dropout(dropout)

    def forward(self, tgt, encoder_output, self_mask=None,
cross_mask=None):
        x = self.embedding(tgt) *
math.sqrt(self.embedding.embedding_dim)
        x = x + self.pos_encoding(x)
        x = self.dropout(x)

        for layer in self.layers:
            x, _ = layer(x, encoder_output, self_mask, cross_mask)

        return x

class Transformer(nn.Module):
    """Full Seq2Seq Transformer model."""

    def __init__(self, vocab_size, hidden_dim=512, num_heads=8,
              num_layers=6, ffn_dim=2048, dropout=0.1):
        super().__init__()
        self.encoder = Encoder(vocab_size, hidden_dim, num_heads,
num_layers, ffn_dim, dropout)
        self.decoder = Decoder(vocab_size, hidden_dim, num_heads,
num_layers, ffn_dim, dropout)
        self.output_layer = nn.Linear(hidden_dim, vocab_size)
        self.vocab_size = vocab_size

    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        encoder_output = self.encoder(src, src_mask)
        decoder_output = self.decoder(tgt, encoder_output, tgt_mask,
src_mask)
        logits = self.output_layer(decoder_output)
        return logits

    def encode(self, src, src_mask=None):
        return self.encoder(src, src_mask)

    def decode(self, tgt, encoder_output, tgt_mask=None,

```

```

src_mask=None):
    return self.decoder(tgt, encoder_output, tgt_mask, src_mask)

    def get_logits(self, decoder_output):
        return self.output_layer(decoder_output)

# Create model
model = Transformer(
    vocab_size=tokenizer.get_vocab_size(),
    hidden_dim=128,
    num_heads=4,
    num_layers=2,
    ffn_dim=256
).to(device)

num_params = sum(p.numel() for p in model.parameters())
print(f"\nTransformer model created")
print(f"Total parameters: {num_params:,}")

Transformer model created
Total parameters: 701,798

```

MODULE 3: Generation Constraints & Quality Metrics

Task 5: Pre-training Loop with Causal Cross-Entropy (2 Marks)

Objective

Train the full network utilizing teacher forcing. Implement standard label smoothing within the cross-entropy loss calculation to prevent the model from becoming overly confident, fostering better generalization during translation/generation.

Label Smoothing

Instead of hard targets [1, 0, 0, ...], use soft targets:

- Correct class: probability 0.9
- Other classes: probability 0.1 / (vocab_size - 1)

Benefits:

- Prevents overconfidence
 - Regularization effect
 - Better generalization
 - Reduced hallucinations
-

Conceptual Explanation

1. Teacher Forcing — Definition and Justification

During training, the decoder at timestep t should receive the ground-truth token y_{t-1} as input regardless of what the model predicted at $t-1$. This technique is called **teacher forcing** (Williams & Zipser, 1989). The alternative — feeding the model's own predictions back as input (free running) — introduces **exposure bias**: errors compound over the decoding steps, and the training distribution drifts away from the inference distribution.

With teacher forcing the decoder input is the target sequence shifted right by one position:

```
decoder_input = [<start>, y_1, y_2, ..., y_{T-1}]
decoder_target = [y_1, y_2, y_3, ..., y_T]
```

The model predicts position t using the true context up to $t-1$, making each cross-entropy term an independent, well-defined classification problem over the vocabulary. This enables fully parallelised training — all timesteps are computed in a single forward pass rather than sequentially.

2. Cross-Entropy Loss with Causal Masking

The per-token cross-entropy loss is:

$$L_{CE} = - (1/T) \sum_{t=1}^T \log p_{\theta}(y_t \mid y_{<t}, x)$$

where p_{θ} is the model's softmax output and x is the source article. Pad tokens must be excluded from the sum — padding positions carry no linguistic signal, and including them would dilute the gradient signal proportionally to the amount of padding in a batch. The implementation masks pad positions by multiplying the per-token loss by a binary indicator ($y_t \neq \text{<pad>}$) and normalising by the number of real tokens.

Gradient clipping (`clip_grad_norm = 1.0`) prevents exploding gradients, which are common in the early training stages of deep Transformers when attention weights have not yet converged to meaningful patterns.

3. Label Smoothing — Formal Description and Effect

Standard cross-entropy trains the model toward a one-hot target distribution:

$$q_{\text{hard}}(k) = 1 \text{ if } k = y_t, \text{ else } 0$$

This encourages the model to assign probability 1 to the correct token and 0 to all others — a state of maximum confidence that is never achievable with finite logits, causing the logits to grow unboundedly during training (overconfidence).

Label smoothing (Szegedy et al., 2016) replaces the hard target with a soft mixture:

$$q_{\text{smooth}}(k) = (1 - \epsilon) \cdot 1[k = y_t] + \epsilon / V$$

where $\epsilon = 0.1$ and V is the vocabulary size. The model is now penalised for assigning *too much* probability mass to the correct class, which:

- **Regularises:** acts as a form of output dropout, preventing the model from memorising training examples.
- **Improves calibration:** the model's confidence scores become meaningful probability estimates rather than saturated near-1 values.
- **Reduces hallucination:** overconfident models tend to generate high-probability tokens from training vocabulary without grounding in the source — label smoothing forces the model to maintain non-zero probability on alternatives, penalising degenerate repetition loops.

Empirically, label smoothing with $\epsilon = 0.1$ consistently improves BLEU and ROUGE scores by 0.5–1.5 points on seq2seq tasks (Vaswani et al., 2017).

```
class LabelSmoothingLoss(nn.Module):
    """
    Cross-entropy loss with label smoothing.

    Prevents model overconfidence by using soft targets.
    """

    def __init__(self, vocab_size, smoothing=0.1, pad_token=0):
        super().__init__()
        self.vocab_size = vocab_size
        self.smoothing = smoothing
        self.pad_token = pad_token
        self.confidence = 1.0 - smoothing

    def forward(self, logits, targets):
        # logits: (batch * seq_len, vocab_size)
        # targets: (batch * seq_len)

        log_probs = F.log_softmax(logits, dim=-1)

        # Create smoothed target distribution
        smooth_targets = torch.full_like(logits,
                                         self.confidence /
                                         (self.vocab_size - 1))
        smooth_targets.scatter_(1, targets.unsqueeze(1),
                               self.confidence)
```

```

# Compute loss
loss = -(smooth_targets * log_probs).sum(dim=1)

# Ignore padding tokens
mask = (targets != self.pad_token).float()
loss = (loss * mask).sum() / mask.sum()

return loss

```

```

class Trainer:
    """Train the model with teacher forcing."""

    def __init__(self, model, optimizer, loss_fn, device):
        self.model = model
        self.optimizer = optimizer
        self.loss_fn = loss_fn
        self.device = device

    def train_step(self, batch):
        self.model.train()

        src = batch['src'].to(self.device)
        tgt = batch['tgt'].to(self.device)
        src_mask = batch['src_mask'].to(self.device)

        # Teacher forcing: feed ground-truth to decoder
        tgt_input = tgt[:, :-1] # Remove last token
        tgt_target = tgt[:, 1:] # Remove first token

        # Create causal mask
        tgt_len = tgt_input.size(1)
        causal_mask = create_causal_mask(tgt_len, self.device)

        # Expand padding mask for attention
        src_mask = src_mask.unsqueeze(1).unsqueeze(1)

        # Forward pass
        logits = self.model(src, tgt_input, src_mask, causal_mask)

        # Compute loss
        logits_flat = logits.view(-1, self.model.vocab_size)
        targets_flat = tgt_target.view(-1)
        loss = self.loss_fn(logits_flat, targets_flat)

        # Backward pass
        self.optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.model.parameters(), 1.0)
        self.optimizer.step()

```

```

        return loss.item()

# Setup training
criteria = LabelSmoothingLoss(vocab_size=tokenizer.get_vocab_size())
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
trainer = Trainer(model, optimizer, criteria, device)

print("TASK 5: Pre-training Loop with Label Smoothing")
print(f"Loss function: Label Smoothing Cross-Entropy")
print(f"Smoothing parameter: 0.1")
print(f"Teacher forcing: Enabled")

TASK 5: Pre-training Loop with Label Smoothing
Loss function: Label Smoothing Cross-Entropy
Smoothing parameter: 0.1
Teacher forcing: Enabled

```

Task 6: Inference Decoding & ROUGE Benchmarking (1.5 Marks)

Objective

Generate summaries for at least five distinct unseen articles from the test set utilizing greedy search decoding. Calculate and report the ROUGE score (ROUGE-1, ROUGE-2, ROUGE-L) and perform a qualitative error analysis evaluating common generation flaws (e.g., repeating word loops, fact hallucinations).

Conceptual Explanation

1. Greedy Search Decoding

At inference time, the decoder generates tokens autoregressively — one token per forward pass. Greedy decoding selects the highest-probability token at each step:

$$\hat{y}_t = \operatorname{argmax}_{v \in V} p_{\theta}(v \mid \hat{y}_{\{<t\}}, x)$$

Greedy decoding is deterministic and $O(T)$ in the number of decoder forward passes. Its primary weakness is **lack of global optimality**: choosing the locally highest-probability token at step t may foreclose higher-probability full sequences. Beam search mitigates this by maintaining k candidate hypotheses, but greedy decoding is the standard baseline for evaluating model quality before search-strategy tuning.

Generation terminates on either the `<end>` token or the maximum length budget — whichever occurs first.

2. ROUGE Metrics — Formal Definitions

ROUGE (Recall-Oriented Understudy for Gisting Evaluation; Lin, 2004) is the standard automatic evaluation suite for summarization. All variants compute overlap between a generated summary (hypothesis H) and one or more human-written references (R).

ROUGE-N measures n-gram overlap. The F1 formulation (used here) balances precision and recall:

$$\begin{aligned}\text{ROUGE-N Precision} &= |\text{ngrams}(H) \cap \text{ngrams}(R)| / |\text{ngrams}(H)| \\ \text{ROUGE-N Recall} &= |\text{ngrams}(H) \cap \text{ngrams}(R)| / |\text{ngrams}(R)| \\ \text{ROUGE-N F1} &= 2 \cdot P \cdot R / (P + R)\end{aligned}$$

- **ROUGE-1** (unigram): measures factual coverage — which key content words appear in the summary.
- **ROUGE-2** (bigram): measures local fluency and semantic coherence — adjacent word pairs that mirror the reference phrasing.

ROUGE-L is based on the Longest Common Subsequence (LCS) between H and R :

$$\begin{aligned}\text{ROUGE-L Precision} &= \text{LCS}(H, R) / \text{len}(H) \\ \text{ROUGE-L Recall} &= \text{LCS}(H, R) / \text{len}(R) \\ \text{ROUGE-L F1} &= 2 \cdot P \cdot R / (P + R)\end{aligned}$$

LCS captures in-sequence n-gram matches without requiring contiguity, making it sensitive to **word order preservation**. A summary that contains all the right words but in scrambled order will score well on ROUGE-1 but poorly on ROUGE-L — an important distinction for coherence evaluation.

3. Qualitative Error Analysis — Taxonomy of Generation Failures

Automatic metrics capture surface-level overlap but do not diagnose *why* a model fails. A systematic error analysis classifies failures into three categories:

Error Type	Mechanism	Detection Method
Repetition loop	Model enters a degenerate state where the highest-probability next token is always the most recently generated token. Common in undertrained models or when the source article provides insufficient guidance.	Bigram repetition count $\geq$ threshold
Hallucination	Model generates named entities, numbers, or facts not grounded in the source article. Arises from high-confidence predictions on frequent training patterns	Source coverage — named-entity overlap between article and summary

Error Type	Mechanism	Detection Method
Length pathology	that override source fidelity. Over-generation (summary longer than reference) indicates the model has not learned the compression objective. Under-generation (summary shorter than <code><end></code> token) indicates the model is not conditioned on the full source.	Generated/reference length ratio

A well-calibrated model should show: repetition rate < 10 %, source coverage > 60 %, and length ratio within [0.8, 1.5] for headline generation tasks.

```
class GreedyGenerator:
    """
    Autoregressive greedy decoder.

    At each step selects argmax over the vocabulary logits.
    Terminates on <end> token (id=2) or max_len, whichever comes
    first.
    """

    def __init__(self, model, tokenizer, device, max_len=30):
        self.model = model
        self.tokenizer = tokenizer
        self.device = device
        self.max_len = max_len

    def generate(self, src_ids):
        """
        Generate a summary token-by-token via greedy search.

        Args:
            src_ids (Tensor): 1-D tensor of source token IDs.
        Returns:
            List[int]: generated token ID sequence (includes <start>,
            may include <end>).
        """
        self.model.eval()
        with torch.no_grad():
            src = src_ids.unsqueeze(0).to(self.device)
            encoder_output = self.model.encode(src)
            summary = [1] # seed with <start> token

            for _ in range(self.max_len):
                summary_tensor = torch.tensor([summary],
dtype=torch.long).to(self.device)
```

```

        causal_mask =
create_causal_mask(summary_tensor.size(1), self.device)
        decoder_output = self.model.decode(summary_tensor,
encoder_output, causal_mask, None)
        logits =
self.model.get_logits(decoder_output[0, -1, :])
        next_token = torch.argmax(logits).item()
        summary.append(next_token)
        if next_token == 2: # <end> token
            break

    return summary

class RougeEvaluator:
    """
    Compute ROUGE-1, ROUGE-2, and ROUGE-L scores (F1 formulation).

    ROUGE-1 : unigram F1 - factual content coverage
    ROUGE-2 : bigram F1 - local phrase coherence
    ROUGE-L : LCS-based F1 - in-order sequence preservation

    All three are reported as F1 to balance precision (summary not
    too verbose) and recall (summary not missing key facts).
    """

    @staticmethod
    def _ngrams(tokens, n):
        """Return a list of n-gram tuples from a token list."""
        return [tuple(tokens[i:i + n]) for i in range(len(tokens) - n
+ 1)]

    @staticmethod
    def _f1(precision, recall):
        """Harmonic mean of precision and recall; returns 0 if both
are 0."""
        if precision + recall == 0:
            return 0.0
        return 2 * precision * recall / (precision + recall)

    @staticmethod
    def _lcs_length(a, b):
        """
        Compute the length of the Longest Common Subsequence of two
        token lists.
        Uses the standard O(|a|*|b|) dynamic-programming algorithm.

        dp[i][j] = LCS length of a[:i] and b[:j]
        """
        m, n = len(a), len(b)

```

```

dp = [[0] * (n + 1) for _ in range(m + 1)]
for i in range(1, m + 1):
    for j in range(1, n + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
return dp[m][n]

def compute_rouge1(self, ref, hyp):
    """ROUGE-1 F1: unigram overlap between hypothesis and
reference."""
    ref_tokens = ref.lower().split()
    hyp_tokens = hyp.lower().split()
    if not ref_tokens or not hyp_tokens:
        return 0.0
    ref_set = set(ref_tokens)
    hyp_set = set(hyp_tokens)
    overlap = len(ref_set & hyp_set)
    precision = overlap / len(hyp_set)
    recall = overlap / len(ref_set)
    return self._f1(precision, recall)

def compute_rouge2(self, ref, hyp):
    """ROUGE-2 F1: bigram overlap between hypothesis and
reference."""
    ref_tokens = ref.lower().split()
    hyp_tokens = hyp.lower().split()
    ref_bigrams = set(self._ngrams(ref_tokens, 2))
    hyp_bigrams = set(self._ngrams(hyp_tokens, 2))
    if not ref_bigrams or not hyp_bigrams:
        return 0.0
    overlap = len(ref_bigrams & hyp_bigrams)
    precision = overlap / len(hyp_bigrams)
    recall = overlap / len(ref_bigrams)
    return self._f1(precision, recall)

def compute_rougeL(self, ref, hyp):
    """
    ROUGE-L F1: Longest Common Subsequence F1.

    Unlike ROUGE-N, LCS does not require contiguous n-gram
matches.
    It rewards in-order word matches across the full sequence,
making
    it sensitive to summary fluency and word-order preservation.

    A summary with all correct words but scrambled order will
score
    well on ROUGE-1 but poorly on ROUGE-L – an important

```

```

distinction
    for coherence evaluation in abstractive summarization.
    """
    ref_tokens = ref.lower().split()
    hyp_tokens = hyp.lower().split()
    if not ref_tokens or not hyp_tokens:
        return 0.0
    lcs = self._lcs_length(ref_tokens, hyp_tokens)
    precision = lcs / len(hyp_tokens)
    recall = lcs / len(ref_tokens)
    return self._f1(precision, recall)

class ErrorAnalyzer:
    """
    Qualitative generation error analysis.

    Diagnoses three systematic failure modes:
    1. Repetition loops - bigram count >= threshold
    2. Length pathology - generated/reference length ratio outside
    [0.5, 2.0]
    3. Low source coverage - hallucination risk indicator
    """

    @staticmethod
    def check_repetition(text, threshold=3):
        """Return True if any bigram appears >= threshold times."""
        words = text.lower().split()
        bigrams = [' '.join(words[i:i + 2]) for i in range(len(words)
- 1)]
        counts = defaultdict(int)
        for bg in bigrams:
            counts[bg] += 1
        return any(c >= threshold for c in counts.values())

    @staticmethod
    def check_length(generated, reference):
        """Return (generated_len, reference_len, ratio)."""
        gen_len = len(generated.split())
        ref_len = len(reference.split())
        ratio = gen_len / ref_len if ref_len > 0 else 0.0
        return gen_len, ref_len, ratio

    @staticmethod
    def compute_coverage(article, summary):
        """Fraction of content words (len > 3) in the article that
appear in the summary."""
        article_words = {w.lower() for w in article.split() if len(w)
> 3}
        summary_words = {w.lower() for w in summary.split()}

```

```

        if not article_words:
            return 0.0
        return len(article_words & summary_words) / len(article_words)

    @staticmethod
    def diagnose(generated, reference, article):
        """
        Return a human-readable diagnosis string for one generated
        summary.
        Checks repetition, length ratio, and source coverage.
        """
        words = generated.lower().split()
        bigrams = [' '.join(words[i:i + 2]) for i in range(len(words)
- 1)]
        counts = defaultdict(int)
        for bg in bigrams:
            counts[bg] += 1
        rep_loops = [bg for bg, c in counts.items() if c >= 3]

        gen_len = len(generated.split())
        ref_len = len(reference.split())
        ratio = gen_len / ref_len if ref_len > 0 else 0.0

        art_words = {w.lower() for w in article.split() if len(w) > 3}
        sum_words = {w.lower() for w in generated.split()}
        coverage = len(art_words & sum_words) / len(art_words) if
art_words else 0.0

        parts = []
        if rep_loops:
            parts.append(f"REPETITION LOOP: {rep_loops[:2]}")
        if ratio < 0.5:
            parts.append(f"UNDER-GENERATION (ratio {ratio:.2f})")
        elif ratio > 2.0:
            parts.append(f"OVER-GENERATION (ratio {ratio:.2f})")
        if coverage < 0.1:
            parts.append(f"HALLUCINATION RISK: low source coverage
({coverage:.0%})")
        return "; ".join(parts) if parts else "No major errors
detected"

print("TASK 6: Inference Decoding & ROUGE Benchmarking")
print(" GreedyGenerator : implemented (autoregressive argmax
decoding)")
print(" RougeEvaluator : ROUGE-1 F1, ROUGE-2 F1, ROUGE-L F1 (LCS-
based)")
print(" ErrorAnalyzer : repetition loop, length ratio, source
coverage, diagnosis")

```

TASK 6: Inference Decoding & ROUGE Benchmarking

GreedyGenerator : implemented (autoregressive argmax decoding)
RougeEvaluator : ROUGE-1 F1, ROUGE-2 F1, ROUGE-L F1 (LCS-based)
ErrorAnalyzer : repetition loop, length ratio, source coverage, diagnosis

Complete Training and Evaluation

— Test corpus (used when CNN/DailyMail is not loaded)

```
test_articles = [  
    "the quick brown fox jumps over the lazy dog in the forest and  
    continues running through the trees until it reaches the river",  
    "artificial intelligence is transforming technology and society by  
    automating tasks that previously required human intelligence",  
    "climate change is affecting weather patterns worldwide causing  
    extreme temperatures droughts and floods in many regions",  
    "renewable energy sources like solar wind and hydropower are  
    becoming increasingly efficient and cost effective",  
    "machine learning algorithms can now recognize images classify  
    text and make predictions with remarkable accuracy",  
]  
  
test_summaries = [  
    "fox jumps over dog",  
    "ai transforms technology",  
    "climate affects weather",  
    "renewable energy improving",  
    "machine learning accurate",  
]
```

— Step 1: Train BPE tokenizer on the full corpus

```
# IMPORTANT: BPETokenizer.train() must be called BEFORE any encode()  
calls.  
# On CNN/DailyMail replace the lists below with:  
# tokenizer.train(train_articles + train_summaries)  
print("Training BPE tokenizer...")  
tokenizer = BPETokenizer(vocab_size=500)  
tokenizer.train(test_articles + test_summaries)
```

— Step 2: Build dataset and DataLoader

```
# Use first 3 articles for training; all 5 articles used during  
evaluation.  
train_dataset = SummarizationDataset(test_articles[:3],
```

```

test_summaries[:3], tokenizer)
loader = DataLoader(train_dataset, batch_size=1,
collate_fn=collate_batch)

# — Step 3: Initialise model with the trained vocabulary size
current_vocab_size = tokenizer.get_vocab_size()
print(f"Vocabulary size after BPE training: {current_vocab_size}")

model = Transformer(
    vocab_size = current_vocab_size,
    hidden_dim = 128,
    num_heads = 4,
    num_layers = 2,
    ffn_dim = 256,
).to(device)

num_params = sum(p.numel() for p in model.parameters())
print(f"Model parameters: {num_params:,}")

# — Step 4: Training loop (teacher forcing + label-smoothing loss)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criteria = LabelSmoothingLoss(vocab_size=current_vocab_size)
trainer = Trainer(model, optimizer, criteria, device)

print("\nTraining for 20 epochs with teacher forcing...")
for epoch in range(20):
    total_loss = 0.0
    for batch in loader:
        total_loss += trainer.train_step(batch)
    if (epoch + 1) % 5 == 0:
        print(f" Epoch {epoch + 1:2d} | Loss = {total_loss:.4f}")

print("Training complete.")

Training BPE tokenizer...
  BPE training complete: 289 merge rules, vocab size = 333
Vocabulary size after BPE training: 333
Model parameters: 790,733

Training for 20 epochs with teacher forcing...
  Epoch 5 | Loss = 6.7656
  Epoch 10 | Loss = 3.0436
  Epoch 15 | Loss = 2.7670
  Epoch 20 | Loss = 2.7453
Training complete.

# — Inference & Evaluation on all 5 test articles

```

```

generator = GreedyGenerator(model, tokenizer, device)
rouge_eval = RougeEvaluator()
error_eval = ErrorAnalyzer()

print("\n" + "=" * 72)
print("TASK 6: EVALUATION ON TEST SET (5 articles)")
print("=" * 72)

rouge1_scores = []
rouge2_scores = []
rougeL_scores = []
coverage_scores = []
repetition_count = 0
error_log = []

for idx, (article, reference) in enumerate(zip(test_articles,
test_summaries), 1):
    cleaned = cleaner.clean(article)
    src_ids = torch.tensor(tokenizer.encode(cleaned))

    # Generate summary via greedy decoding
    summary_ids = generator.generate(src_ids)
    # Strip <start>=1 and <end>=2 special tokens before decoding to
text
    content_ids = [t for t in summary_ids if t not in (1, 2)]
    generated = tokenizer.decode(content_ids)

    # — ROUGE scores (all three, F1 formulation)


---


    r1 = rouge_eval.compute_rouge1(reference, generated)
    r2 = rouge_eval.compute_rouge2(reference, generated)
    rL = rouge_eval.compute_rougeL(reference, generated)
    rouge1_scores.append(r1)
    rouge2_scores.append(r2)
    rougeL_scores.append(rL)

    # — Error analysis


---


    has_rep = error_eval.check_repetition(generated)
    gen_len, ref_len, ratio = error_eval.check_length(generated,
reference)
    coverage = error_eval.compute_coverage(cleaned,
generated)
    diagnosis = error_eval.diagnose(generated, reference,
cleaned)
    coverage_scores.append(coverage)
    if has_rep:
        repetition_count += 1
    error_log.append((idx, diagnosis))

```

```

print(f"\nArticle {idx}")
print(f"  Source      : {article[:70]}...")
print(f"  Reference   : {reference}")
print(f"  Generated    : {generated}")
print(f"  ROUGE-1 F1: {r1:.3f} | ROUGE-2 F1: {r2:.3f} | ROUGE-
L F1: {rL:.3f}")
print(f"  Length      : generated={gen_len} words,
reference={ref_len} words, ratio={ratio:.2f}")
print(f"  Coverage    : {coverage:.1%}")
print(f"  Diagnosis   : {diagnosis}")

```

— Aggregate ROUGE results

```

print("\n" + "=" * 72)
print("AGGREGATE ROUGE SCORES (F1, averaged over 5 articles)")
print("=" * 72)
print(f"  ROUGE-1 F1 : {np.mean(rouge1_scores):.3f}")
print(f"  ROUGE-2 F1 : {np.mean(rouge2_scores):.3f}")
print(f"  ROUGE-L F1 : {np.mean(rougeL_scores):.3f}")
print(f"  Avg Coverage : {np.mean(coverage_scores):.1%}")
print(f"  Repetition   : {repetition_count}/{len(test_articles)}
articles affected")

```

— Qualitative error analysis summary

```

print("\n" + "=" * 72)
print("QUALITATIVE ERROR ANALYSIS")
print("=" * 72)
print("""
Three systematic failure modes are evaluated for each generated
summary:

```

1. REPETITION LOOPS

Cause : The model enters a degenerate probability peak where the highest-probability next token is always the last generated token. Common in undertrained models or when cross-attention weights have not converged to stable source-side alignment.

Signal : Any bigram (two-word phrase) appears ≥ 3 times in output.

Remedy : (a) More training epochs, (b) repetition penalty during decoding, (c) minimum-length constraint to prevent early <end> collapse.

2. HALLUCINATION

Cause : The model generates tokens with high marginal probability from training distribution but not grounded in the source article. Overconfident models (without label smoothing)

```
are
    especially susceptible – they memorise high-frequency
    training phrases and reproduce them regardless of source.
Signal  : Source coverage < 10% (summary contains almost no content
          words that appear in the article).
Remedy  : (a) Increase label smoothing epsilon, (b) copy/pointer
          mechanism, (c) faithfulness-constrained decoding.
```

3. LENGTH PATHOLOGY

```
Cause   : Under-generation – model predicts <end> prematurely,
          often because <end> has high marginal probability early
          in training. Over-generation – model never predicts
<end>,
          usually from insufficient training or misconfigured
max_len.
Signal  : Length ratio < 0.5 (under) or > 2.0 (over).
Remedy  : Length penalty in loss function, explicit min/max length
          constraints during decoding.
```

```
""")
```

```
print("Per-article diagnosis:")
```

```
for idx, diag in error_log:
    print(f" Article {idx}: {diag}")
```

```
print("\nNote: Low ROUGE scores are expected – this is a small demo
model")
```

```
print("trained on 3 toy sentences for 20 epochs. On CNN/DailyMail
with")
```

```
print("full training, ROUGE-1 ~ 30-44 is achievable (BART benchmark:
44.16).")
```

```
=====
==
TASK 6: EVALUATION ON TEST SET (5 articles)
=====
==
```

Article 1

```
Source   : the quick brown fox jumps over the lazy dog in the
forest and continue...
```

```
Reference : fox jumps over dog
```

```
Generated : fox jumps over dog
```

```
ROUGE-1 F1: 1.000 | ROUGE-2 F1: 1.000 | ROUGE-L F1: 1.000
```

```
Length    : generated=4 words, reference=4 words, ratio=1.00
```

```
Coverage  : 15.4%
```

```
Diagnosis : No major errors detected
```

Article 2

```
Source   : artificial intelligence is transforming technology and
```

society by auto...

Reference : ai transforms technology

Generated : ai transforms technology

ROUGE-1 F1: 1.000 | ROUGE-2 F1: 1.000 | ROUGE-L F1: 1.000

Length : generated=3 words, reference=3 words, ratio=1.00

Coverage : 11.1%

Diagnosis : No major errors detected

Article 3

Source : climate change is affecting weather patterns worldwide causing extreme...

Reference : climate affects weather

Generated : climate affects weather

ROUGE-1 F1: 1.000 | ROUGE-2 F1: 1.000 | ROUGE-L F1: 1.000

Length : generated=3 words, reference=3 words, ratio=1.00

Coverage : 15.4%

Diagnosis : No major errors detected

Article 4

Source : renewable energy sources like solar wind and hydropower are becoming i...

Reference : renewable energy improving

Generated : climate affects weather

ROUGE-1 F1: 0.000 | ROUGE-2 F1: 0.000 | ROUGE-L F1: 0.000

Length : generated=3 words, reference=3 words, ratio=1.00

Coverage : 0.0%

Diagnosis : HALLUCINATION RISK: low source coverage (0%)

Article 5

Source : machine learning algorithms can now recognize images classify text and...

Reference : machine learning accurate

Generated : climate affects weather

ROUGE-1 F1: 0.000 | ROUGE-2 F1: 0.000 | ROUGE-L F1: 0.000

Length : generated=3 words, reference=3 words, ratio=1.00

Coverage : 0.0%

Diagnosis : HALLUCINATION RISK: low source coverage (0%)

=====

==
AGGREGATE ROUGE SCORES (F1, averaged over 5 articles)

=====

==
ROUGE-1 F1 : 0.600

ROUGE-2 F1 : 0.600

ROUGE-L F1 : 0.600

Avg Coverage : 8.4%

Repetition : 0/5 articles affected

=====

==
QUALITATIVE ERROR ANALYSIS
=====

Three systematic failure modes are evaluated for each generated summary:

1. REPETITION LOOPS

Cause : The model enters a degenerate probability peak where the highest-probability next token is always the last generated token. Common in undertrained models or when cross-attention weights have not converged to stable source-side alignment.
Signal : Any bigram (two-word phrase) appears ≥ 3 times in output.
Remedy : (a) More training epochs, (b) repetition penalty during decoding, (c) minimum-length constraint to prevent early `<end>` collapse.

2. HALLUCINATION

Cause : The model generates tokens with high marginal probability from training distribution but not grounded in the source article. Overconfident models (without label smoothing) are especially susceptible – they memorise high-frequency training phrases and reproduce them regardless of source.
Signal : Source coverage $< 10\%$ (summary contains almost no content words that appear in the article).
Remedy : (a) Increase label smoothing epsilon, (b) copy/pointer mechanism, (c) faithfulness-constrained decoding.

3. LENGTH PATHOLOGY

Cause : Under-generation – model predicts `<end>` prematurely, often because `<end>` has high marginal probability early in training. Over-generation – model never predicts `<end>`, usually from insufficient training or misconfigured `max_len`.
Signal : Length ratio < 0.5 (under) or > 2.0 (over).
Remedy : Length penalty in loss function, explicit min/max length constraints during decoding.

Per-article diagnosis:

Article 1: No major errors detected
Article 2: No major errors detected
Article 3: No major errors detected
Article 4: HALLUCINATION RISK: low source coverage (0%)

Article 5: HALLUCINATION RISK: low source coverage (0%)

Note: Low ROUGE scores are expected – this is a small demo model trained on 3 toy sentences for 20 epochs. On CNN/DailyMail with full training, ROUGE-1 ~ 30-44 is achievable (BART benchmark: 44.16).

Summary

This notebook implements all 6 tasks required for the Sequence-to-Sequence Transformer assignment:

Task 1: Content Truncation & Cleaning (1.5 marks)

- Text normalisation: HTML removal, non-ASCII stripping, whitespace collapsing
- Metadata filtering: timestamps, author by-lines, section labels
- Inverted-pyramid truncation: 60% lead + 30% sampled middle + 10% tail
- Conceptual justification: $O(n^2)$ attention cost, encoder-decoder vs decoder-only for asymmetric lengths

Task 2: Subword Tokenization & Dynamic Padding (1.5 marks)

- Full BPE algorithm trained from scratch (merge-rule learning loop)
- `</w>` end-of-word marking; graceful OOV decomposition to characters
- Dynamic padding via `collate_batch` — batch-level max length, not global
- Conceptual justification: OOV problem, BPE formal algorithm, dynamic vs global padding

Task 3: Encoder-Decoder & Cross-Attention (2 marks)

- Multi-head self-attention from scratch (Q/K/V projections, scaled dot-product)
- Bidirectional encoder stack with sinusoidal positional encoding
- Decoder with masked self-attention + cross-attention to encoder output
- Conceptual justification: attention math, multi-head diversity, information bottleneck

Task 4: Causal & Sequence Masking (1.5 marks)

- Padding mask (`batch, 1, 1, src_len`) — prevents attention to pad tokens
- Causal lower-triangular mask (`1, 1, tgt_len, tgt_len`) — prevents future leakage
- Combined decoder mask: element-wise AND of both masks
- Conceptual justification: gradient leakage from padding, exposure bias from future tokens

Task 5: Pre-training Loop (2 marks)

- Teacher forcing: decoder input = target shifted right
- Label smoothing cross-entropy ($\epsilon=0.1$): prevents overconfidence
- Gradient clipping (norm=1.0), pad-token masking in loss
- Conceptual justification: teacher forcing vs free-running, label smoothing math

Task 6: Inference & ROUGE Evaluation (1.5 marks)

- Greedy search decoding: argmax at each timestep, terminates on `<end>`
- **ROUGE-1 F1**: unigram overlap (factual coverage)
- **ROUGE-2 F1**: bigram overlap (local phrase coherence)
- **ROUGE-L F1**: LCS-based (word-order preservation) — implemented from scratch with $O(|H| \cdot |R|)$ DP
- Qualitative error analysis: repetition loops, hallucination (source coverage), length pathology
- Evaluation on 5 test articles with per-article diagnosis and aggregate scores
- Conceptual justification: greedy vs beam, ROUGE metric definitions, error taxonomy